

A Report on Secure Programming in a Static Website according to OWASP

Bruno Bucalon Serra
Department of Computer Science
University of London

CONTENTS

I	Introduction	1
II	Criteria for Guideline Selection	1
III	Analysis and Interpretation	1
III-A	Item 11	1
III-B	Item 13	1
III-C	Item 207	1
III-D	Item 209	2
III-E	Item 212	2
IV	Conclusion	2
	References	2

LIST OF TABLES

I	Summary of what is the problem, what should change and how	1
---	--	---

A Report on Secure Programming in a Static Website according to OWASP

Abstract—This report presents an analysis of secure programming techniques for the Software Design and Development module of the BSc Computer Science programme at the University of London, with Goldsmiths direction, regarding secure programming techniques using OWASP coding practices. It reviews a project of a data visualization tool developed for Introduction to Programming Language II course during the first two quarters of 2025.

I. INTRODUCTION

The security of a software has the goal to maintain the confidentiality, integrity, and availability of information resources in order to enable successful business operations [1]. Consequently, OWASP (Open Worldwide Application Security Project), a global non-profit organization, created the Secure Coding Practices Quick Reference Guide with 202 suggestions organized into 14 top level areas (e.g. Input Validation).

Those practices will be used to evaluate a data visualization tool developed using JavaScript with p5 library, as well as HTML and CSS. The application runs in standard web browsers such as Mozilla Firefox and Google Chrome. It processes CSV files stored in project directories organized by topic, including climate change, food attitudes, and AI usage and converts the data into interactive visualizations.

II. CRITERIA FOR GUIDELINE SELECTION

For the project analysis, five suggestions were selected from the Secure Coding Practices Quick Reference Guide from General Coding Practices and Input Validation top areas. These two sections were selected because the project under review is primarily a front-end website developed with the p5.js library from JavaScript, and does not include backend components, therefore the two selected areas are the most relevant for the analysis.

III. ANALYSIS AND INTERPRETATION

This section will be focused on explaining how each item could be used in secure programming techniques seen in Software Design course to ensure or check compliance with each item. The table below summarizes all items, which are subsequently examined and discussed individually. Each subsection describes the specific outputs in detail and explains how they were derived. For each item, the corresponding analysis outputs can be evaluated.

TABLE I. SUMMARY OF WHAT IS THE PROBLEM, WHAT SHOULD CHANGE AND HOW

Item	What Problem	What should change	How should change
Item 11	No type validation in calculations	Validate input data	Use <code>typeof</code> checks
Item 13	Unexpected data length	Limit array sizes	Validate lengths before loops
Item 207	Uninitialized parameters	Ensure initialization	Set defaults or validate inputs
Item 209	Division by zero risk	Handle empty datasets	Check length before division
Item 212	Third-party code risk	Review dependencies	Monitor official library updates

A. Item 11

This item refers to performing data validations in core data structures and variables used in methods and functions. Throughout the application, functions were created to assist data calculations to pre-process the data and plot into charts [1]. These functions are:

- `sum`, to sum the values from selected rows.
- `mean`, to calculate the mean from selected rows.

The problem in these two functions is no data validations is performed, which could lead to runtime type errors and breaking visualizations. This could be avoided by applying in both of the functions the `typeof` operator from JavaScript together with conditional logic could be used to check whether any invalid data were collected is different from a number.

B. Item 13

This item addresses the validation of unexpected data length. It may rise from errors in data parsing, incomplete CSV, future changes in dataset or even bugs from library [1].

In the code base, the function `sliceRowNumbers` extracts numeric values from a data row and returns them as an array, it uses `start` and `end` parameters. Moreover, `drawYAxisTickLabels` draws the labels and grid lines on the Y-axis of the chart, it's controlled by `numYTickLabels` that defines how many tick labels are draw in Y axis.

Both of the problems could lead to massive loops that negatively affect rendering. To mitigate these issues, verify that array lengths are according to the expected limits before the processing is crucial in order to avoid consumption of resources, unnecessary loops and invalid results.

C. Item 207

The item 207 states that the developers should explicitly initialize all your variables and other data stores, either during declaration or just before the first usage [1].

This item is partially attended in the data visualization tool, since some functions properly initialize their variables, such as `sum`, that performs `var total = 0`.

However, the problem appears in `sliceRowNumbers` function, that, for example, has three parameters: `row`, `start`, and `end`. To be appropriate to the item, `start` could be initialized with 0 (`start=0`) to prevent undefined from affecting the loop. Nevertheless, initialize `end` would be a guess since the dataset length depends in which CSV is used, and `row` cannot work without a valid row, thus it should be validated, not initialized.

D. Item 209

Avoid calculation errors by understanding your programming language's underlying representation and how it interacts with numeric calculation. Pay close attention to byte size discrepancies, precision, signed/unsigned distinctions, truncation, conversion and casting between types, "not-a-number" calculations, and how your language handles numbers that are too large or too small for its underlying representation [1].

This item is also applied in the `sum` function as in item 11.

The calculation of `mean` is done by dividing the `total` by the `data.length`, therefore, if the length is zero, an infinity or NaN (Not a Number) would be set as result. To mitigate it, an exception could be created, to define the dataset as empty and not generating floating-point values used to represent results that are outside the range of normal real numbers or are mathematically undefined:

```
if (data.length === 0) {
  throw new Error("Empty dataset");
}
```

E. Item 212

Finally, the item 212 states that to review all secondary applications, third party code and libraries to determine business necessity and validate safe functionality, as these can introduce new vulnerabilities [1].

It is applied by restricting the project to well-known third-party libraries, such as `p5.js`, sourced from official distributions, and by avoiding unnecessary dependencies, reducing the attack surface and maintenance risk.

However, as an additional review, new releases from GitHub can be monitored by setting the Watch option on the repository to receive notifications when updates are published. This allows each new feature to be reviewed in order to prevent potential issues in the codebase.

However, in the context of this project, which is primarily a front-end website developed with the `p5.js` library from JavaScript and does not include backend components, most of the items from OWASP couldn't be applied. This limitation is because the biggest portion of OWASP primarily target backend-related concerns and possible issues.

REFERENCES

- [1] OWASP Foundation, "OWASP Secure Coding Practices – Quick Reference Guide," version 2.1, 2021. Available: https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/assets/docs/OWASP_SCP_Quick_Reference_Guide_v21.pdf.

IV. CONCLUSION

The usage of OWASP coding practice can significantly improve the code base if properly implemented. The Input Validation area is practical and straightforward to apply, and could help prevent critical errors in the codebase. In particular, items 11, 13, and 207, that check functions and parameters to avoid the program failure during its execution.